Assignment-1

Student Name: Taran Mamidala **UB Person Number:** 50604177

Student Name: Tarun Jangam **UB Person Number:** 50600982

PART-1: Data Analysis & Preprocessing

1. **Provide short details of the methods you used for data preprocessing.**

   Below are some methods that we have used in this part for preprocessing of data

   Handling Missing values.
   Removing duplicates.
   Dropping/ Impute rows for missing values of string data types.
   Replacing missing values with mean.
   Handling mismatched formats.
   Detect and handling outliers.
   Impute the outliers with nearest value.
   Converting features into categorical variables.
   Feature selection.
   Normalizing the data.
   Data Splitting.

2. **For each dataset (three in total):**

   a. **Provide a brief overview of your dataset. Describe its domain, type of data, number of samples, and features.**

      Penguin Dataset:

      The dataset focuses on different species of penguins and their characteristics, collected from various islands. It contains 344 individual records, which can help analyze and predict specific traits like the penguin's gender or the island it was found on.

Key Details:

Species: The penguins belong to one of three species – Adelie, Chinstrap, or Gentoo.
Island: The dataset captures which island the penguin was found on either Torgersen, Dream, or Biscoe.
Caloric Requirement: The daily caloric needs of the penguin in calories.
Average Sleep Duration: The average number of hours each penguin sleeps.
Bill Length & Depth: The size of the penguin's bill, measured in millimeters.
Flipper Length: The length of the penguin's flippers, also in millimeters.
Body Mass: The penguin's body mass, measured in grams.
Gender: The sex of the penguin, either male or female.
Year of Observation: The year the data was recorded.

Purpose:

The dataset can be used to predict:

Gender: Using the penguin's characteristics to identify whether it is male or female.
Island: Predicting the island a penguin was found on based on its physical traits and other details.

Data Types:

Categorical data include features like Species , Island, Gender

Numerical data include features like Caloric Requirement , Sleep Duration, Bill Measurements, Flipper Length, Body Mass, Year

Breeding Bird Atlas Dataset:

The Breeding Bird Atlas dataset provides detailed observations on bird breeding behaviors and protection statuses within a specific region. The dataset includes 361,582 entries, making it a robust resource for analyzing bird specie's breeding patterns and the impact of environmental factors.

Key Features:

Fed. Region: The federal region where the observation was recorded.
Block ID: Unique identifier for the observation block.
Map Link: Link to the map showing the location of the observation.
County: The county where the breeding behavior was noted.

Common Name & Scientific Name: Both the common and scientific names of the bird species.
NYS Protection Status: Indicates whether the species is protected by New York State laws.
Family Name & Description: The bird's broader family classification and details about that family group.
Breeding Behavior: Notes on the observed breeding behavior of the bird.
Month, Day, Year: The exact date of the observation.
Temperature: The temperature at the time the breeding activity was observed.

The **"Breeding Status"** is the target variable of interest, indicating whether or not the bird is breeding. Another important variable is the **"NYS Protection Status"**, which shows whether the species is protected by state regulations.

Data Types:

Categorical data includes features like region, county, species names, protection status, and breeding behavior.
Numerical data covers variables such as the date such as month, day ,year and temperature.

Diamond Dataset:

The dataset consists of 53,940 entries, each row represents a unique diamond, with several features describing its physical properties, quality, and market context.

Key Features:

carat: The weight of the diamond.
cut: The quality of the diamond's cut.
color: The diamond's color grade.
clarity: The clarity grade, indicating how flawless the diamond is.
average US salary: Economic context, potentially influencing diamond prices.
number of diamonds mined (millions): Global diamond production.
depth: The depth percentage of the diamond.
table: The width of the diamond's top surface as a percentage of its average diameter.
price: The market value of the diamond ,**this is the variable we aim to predict**.
x, y, z: The diamond's physical dimensions like length, width, and height.

Data Types:

Categorical data includes features like cut, color, and clarity ,etc.

Numerical data includes features like carat, average US salary, number of diamonds mined, depth, table, price, and the dimensions x, y, z

b. Present key statistics (e.g. mean, standard deviation, number of missing values for each feature)

Penguin Dataset:

Dataset Information

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 344 entries, 0 to 343
Data columns (total 10 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   species                 333 non-null    object
 1   island                  334 non-null    object
 2   calorie requirement     344 non-null    int64
 3   average sleep duration  344 non-null    int64
 4   bill_length_mm          337 non-null    float64
 5   bill_depth_mm           333 non-null    float64
 6   flipper_length_mm       336 non-null    float64
 7   body_mass_g             339 non-null    float64
 8   gender                  327 non-null    object
 9   year                    342 non-null    float64
dtypes: float64(5), int64(2), object(3)
memory usage: 27.0+ KB
```

Dataset Statistics

```
       calorie requirement  average sleep duration  bill_length_mm  \
count           344.000000              344.000000      337.000000
mean           5270.002907               10.447674       45.494214
std            1067.959116                2.265895       10.815787
min            3504.000000                7.000000       32.100000
25%            4403.000000                9.000000       39.500000
50%            5106.500000               10.000000       45.100000
75%            6212.750000               12.000000       49.000000
max            7197.000000               14.000000      124.300000

       bill_depth_mm  flipper_length_mm  body_mass_g         year
count     333.000000         336.000000   339.000000   342.000000
mean       18.018318         197.764881  4175.463127  2008.035088
std         9.241384          27.764491   858.713267     0.816938
min        13.100000          10.000000   882.000000  2007.000000
25%        15.700000         190.000000  3550.000000  2007.000000
50%        17.300000         197.000000  4050.000000  2008.000000
75%        18.700000         213.000000  4750.000000  2009.000000
max       127.260000         231.000000  6300.000000  2009.000000
```

Number of missing values for each feature

	0
species	11
island	10
calorie requirement	0
average sleep duration	0
bill_length_mm	7
bill_depth_mm	11
flipper_length_mm	8
body_mass_g	5
gender	17
year	2

dtype: int64

Diamond Dataset

Dataset Information

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 53940 entries, 0 to 53939
Data columns (total 12 columns):
 #   Column                              Non-Null Count  Dtype
---  ------                              --------------  -----
 0   carat                               52430 non-null  object
 1   cut                                 52647 non-null  object
 2   color                               52428 non-null  object
 3   clarity                             53587 non-null  object
 4   average us salary                   53940 non-null  int64
 5   number of diamonds mined (millions) 53940 non-null  float64
 6   depth                               53246 non-null  object
 7   table                               52398 non-null  object
 8   price                               52357 non-null  object
 9   x                                   52414 non-null  object
 10  y                                   52719 non-null  object
 11  z                                   52507 non-null  object
dtypes: float64(1), int64(1), object(10)
memory usage: 4.9+ MB
```

Dataset Statistics

	average us salary	number of diamonds mined (millions)
count	53940.000000	53940.000000
mean	39521.990100	2.902669
std	5486.892971	1.325985
min	30000.000000	0.600000
25%	34780.000000	1.750000
50%	39547.500000	2.910000
75%	44252.000000	4.050000
max	48999.000000	5.200000

Number of missing values for each feature

	0
Unnamed: 0	377
carat	1510
cut	1293
color	1512
clarity	353
average us salary	0
number of diamonds mined (millions)	0
depth	694
table	1542
price	1583
x	1526
y	1221
z	1433

dtype: int64

Breeding Bird Atlas Dataset:

Dataset Information

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 361582 entries, 0 to 361581
Data columns (total 16 columns):
 #   Column                 Non-Null Count   Dtype  
---  ------                 --------------   -----  
 0   Fed. Region            355787 non-null  float64
 1   Block ID               358864 non-null  object 
 2   Map Link               356865 non-null  object 
 3   County                 350980 non-null  object 
 4   Common Name            351052 non-null  object 
 5   Scientific Name        354097 non-null  object 
 6   NYS Protection Status  353112 non-null  object 
 7   Family Name            359126 non-null  object 
 8   Family Description     356849 non-null  object 
 9   Breeding Behavior      356399 non-null  object 
 10  Month                  3426 non-null    float64
 11  Day                    9338 non-null    float64
 12  Year                   351102 non-null  float64
 13  Temperature            361582 non-null  int64  
 14  Average UB Student GPA 361582 non-null  float64
 15  Breeding Status        361582 non-null  object 
dtypes: float64(5), int64(1), object(10)
memory usage: 44.1+ MB
```

Dataset Statistics

	Fed. Region	Month	Day	Year	Temperature	Average UB Student GPA
count	95150.000000	912.000000	2483.000000	93979.000000	96700.000000	96700.000000
mean	5.617267	48.575658	49.076118	1965.646410	49.550445	2.850498
std	5.703018	28.554314	28.730237	190.815055	17.325377	0.490884
min	0.000000	0.000000	0.000000	0.000000	20.000000	2.000000
25%	4.000000	24.000000	24.000000	1984.000000	35.000000	2.430000
50%	5.000000	49.000000	48.000000	1985.000000	50.000000	2.850000
75%	7.000000	72.000000	74.000000	1985.000000	65.000000	3.272500
max	99.000000	99.000000	99.000000	1985.000000	79.000000	3.700000

Number of missing values for each feature

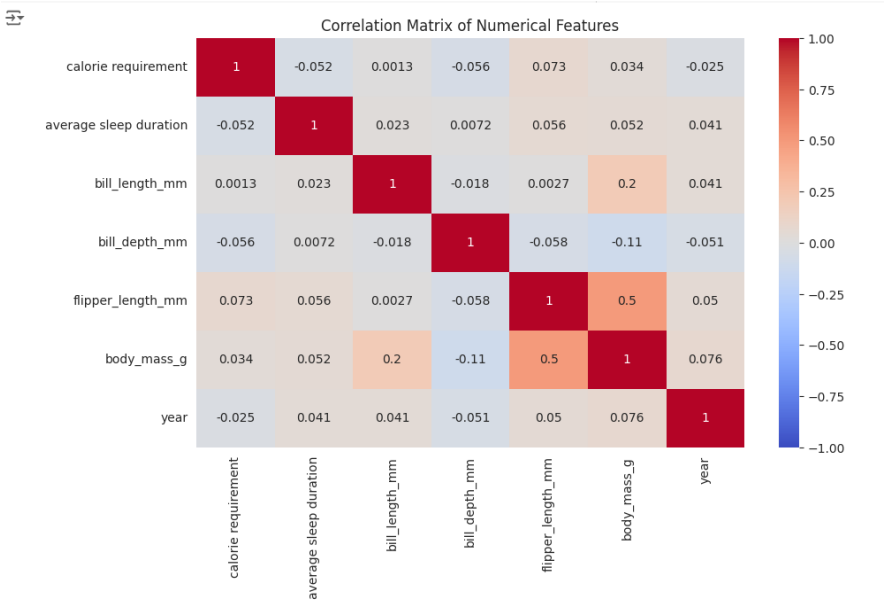
	0
Fed. Region	5795
Block ID	2718
Map Link	4717
County	10602
Common Name	10530
Scientific Name	7485
NYS Protection Status	8470
Family Name	2456
Family Description	4733
Breeding Behavior	5183
Month	358156
Day	352244
Year	10480
Temperature	0
Average UB Student GPA	0
Breeding Status	0

dtype: int64

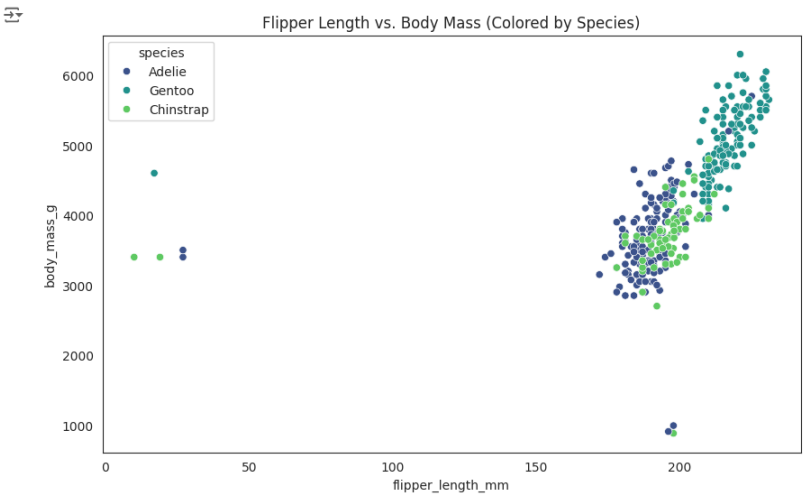
c. Include at least 5 graphs, such as histograms, scatter plots, or correlation matrices. Briefly describe the insights gained from these visualizations.

Penguin Dataset:

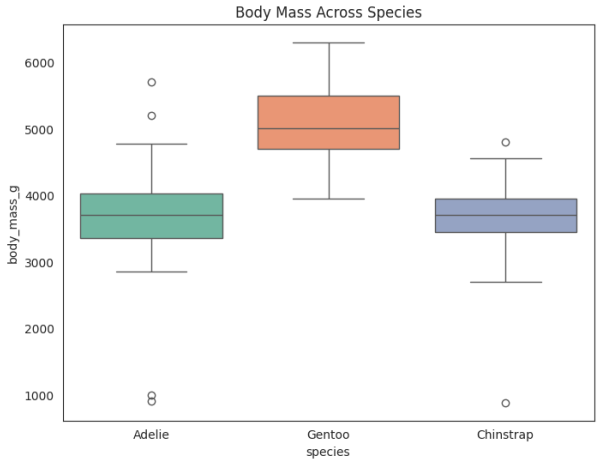
Correlation matrix for numerical features using Heatmap



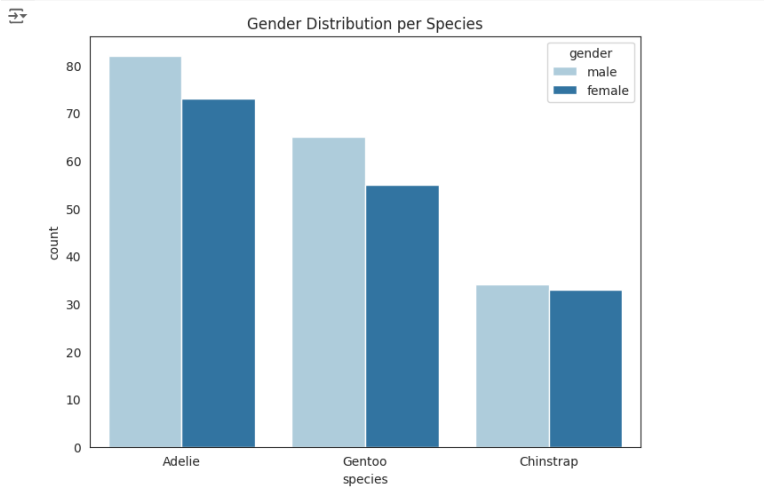
Scatter plot of Flipper Length vs Body Mass in g by Type of Species



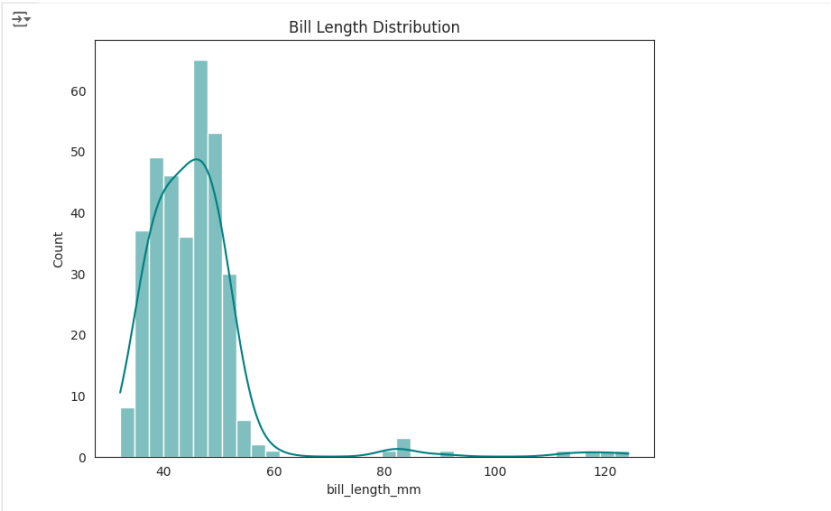
Box plot of Average Body Masses based on Species



Gender Distribution per Species



Bill Length Distribution



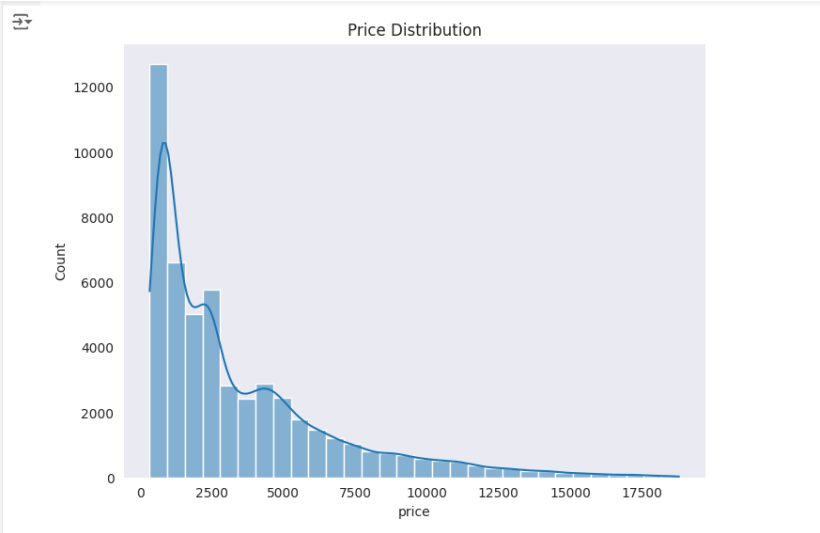
Insights Gained from Visualizations:

- The plot shows larger penguins have longer flippers, with Gentoo being the biggest species.
- The majority of the penguins belong to the **ADELIE** species.
- A scatter plot helps identify the heavier and lighter penguins based on their species
- The **ADELIE** species has notably shorter bill lengths compared to other species.
- The chart shows a slightly higher number of males than females in all penguin species, with the closest balance in Chinstrap penguins.

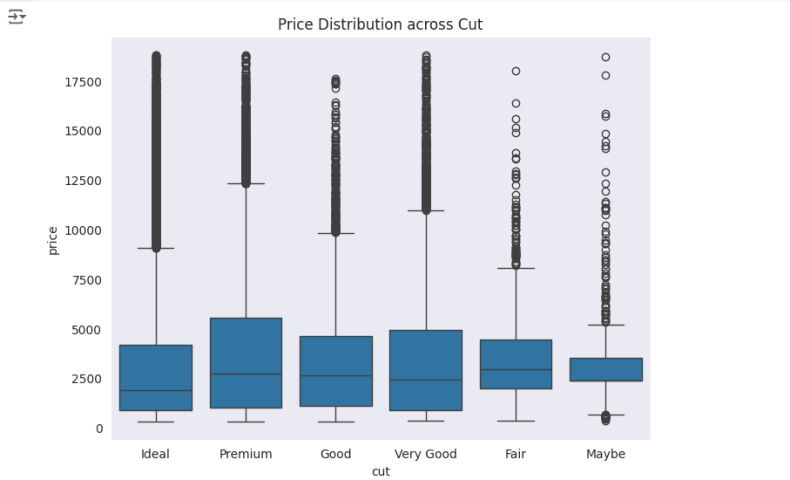
Diamond Dataset:

Carat vs Price

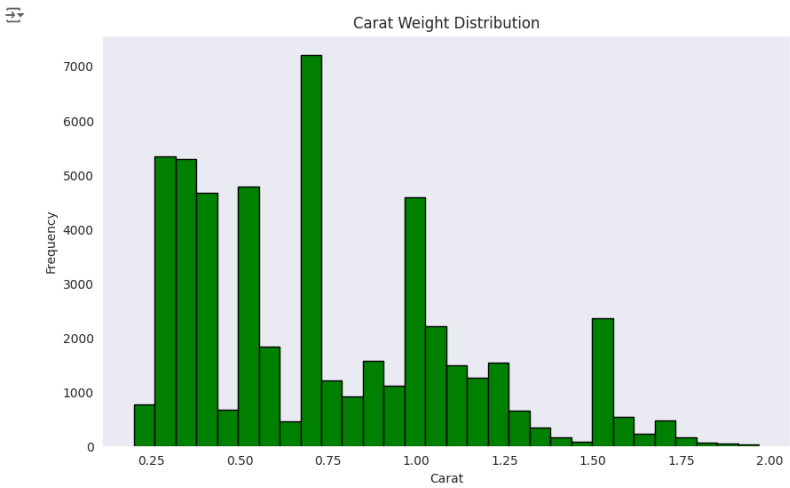
Distribution of Diamond Prices



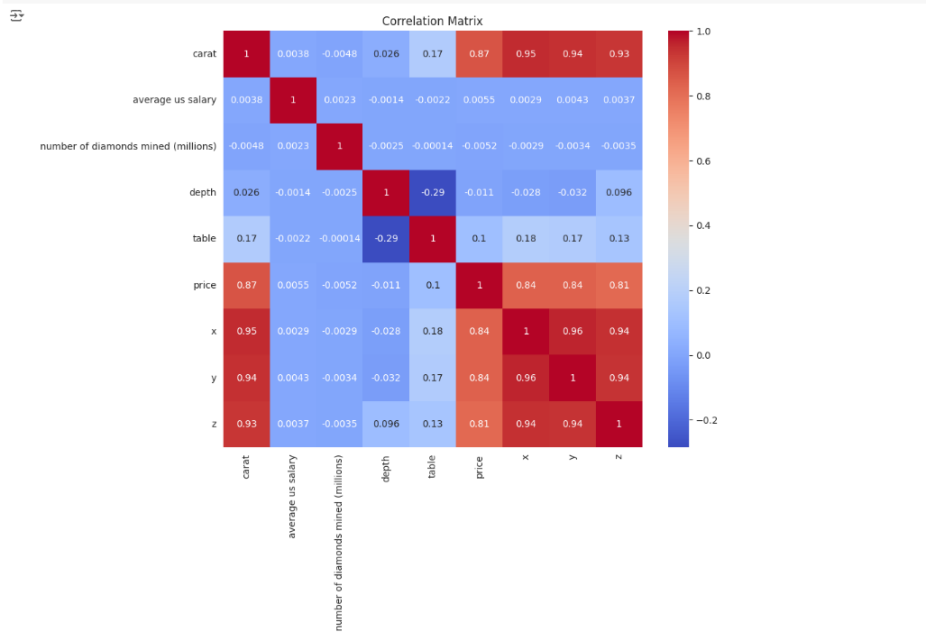
Price Distribution across Cut



Carat Weight Distribution



Correlation matrix for numerical features using Heatmap

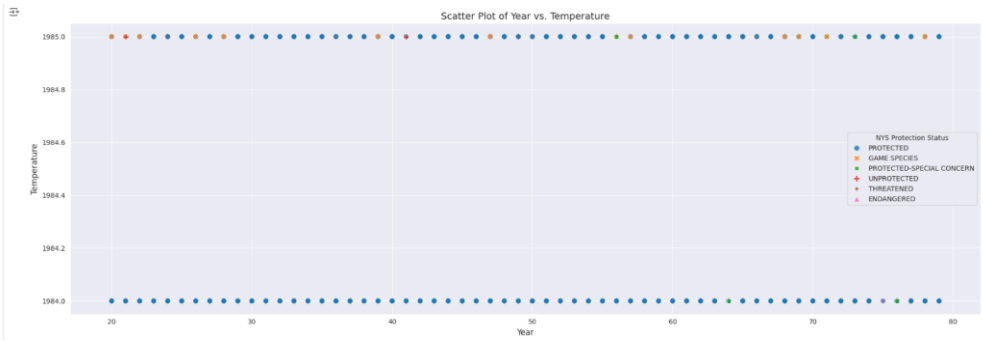


Insights Gained from Visualizations:

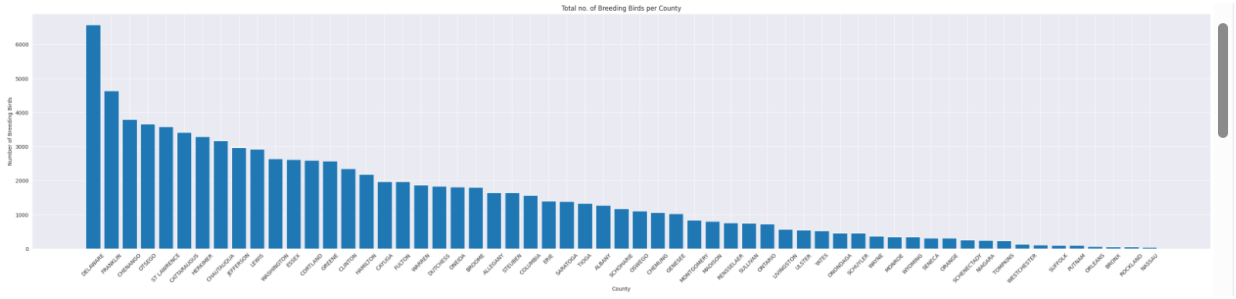
- The boxplot shows that diamonds with an “Ideal” cut have the widest price range, while the “Maybe” cut has the lowest median price and a narrower distribution compared to others.
- The histogram shows that most diamonds are concentrated around the 0.75 carat mark, with fewer diamonds at both lower and higher weights.
- If we look at a scatter plot, we can see how diamond prices vary based on their carat weight and cut quality.
- Generally, as the price of diamonds goes up, their availability tends to decrease.
- There’s a moderate correlation between the depth and table measurements of diamonds.
- Diamonds with an ideal or premium cut typically command higher prices.

Breeding Bird Atlas Dataset:

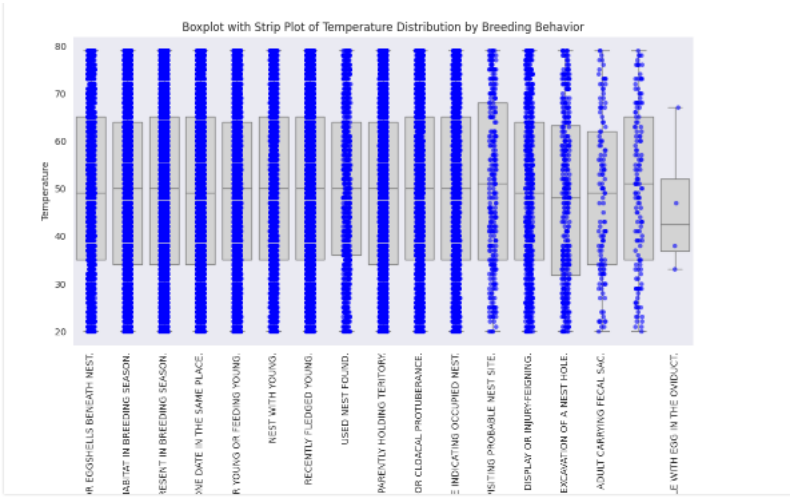
Temperature vs. Year



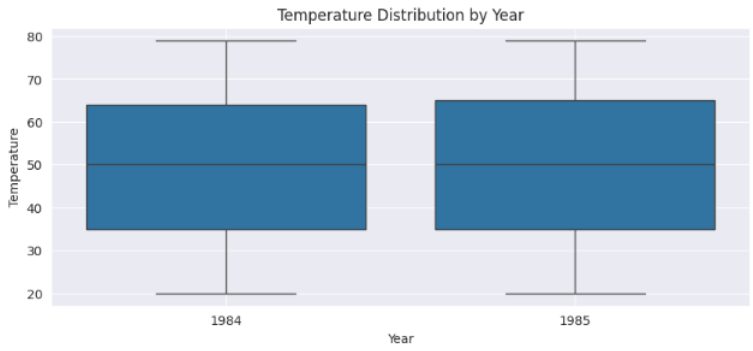
Total number of breeding bird per County



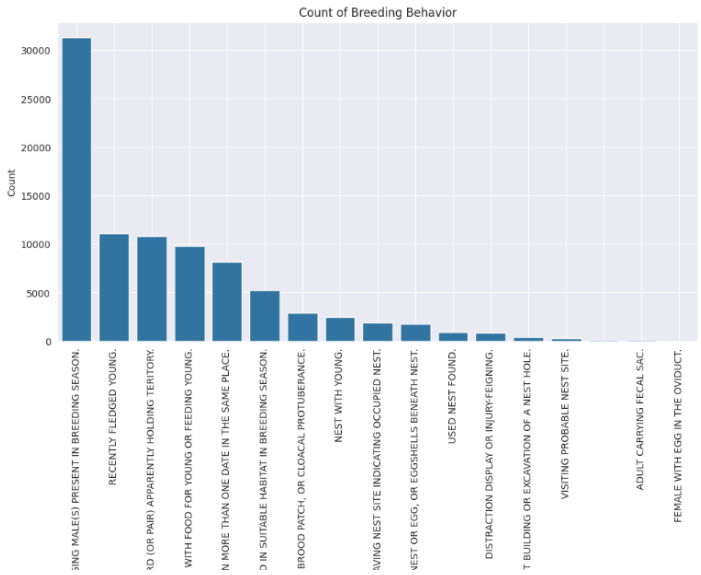
Distribution of Temperature by Breeding Behavior



Box plot of Temperature Distribution by Year



Bar plot for Count of Breeding Behavior



Insights Gained from Visualizations:

- The number of breeding birds is highest in St. Lawrence County.
- A scatter plot of temperature vs. year based on NYS Protection status reveals temperature trends over time, providing valuable insights for understanding climate changes and their impact on bird conservation efforts.

- Higher temperatures are observed during the breeding period when female birds have eggs in the oviduct.
- The box plot highlights temperature variations across different years.
- There are more sightings of species in potential nesting habitats or of singing males during the breeding season.

PART-2: Logistic Regression using Gradient Descent

We have achieved an Best Test accuracy of 81.16% and Best Training Accuracy of 81.16% using Logistic Regression Method

Test Accuracy

```
# Predicting on test set and calculating accuracy
y_pred = model.predict(X_test)
accuracy = np.mean(y_pred == y_test)
print(f'Accuracy: {accuracy * 100:.2f}%')
```

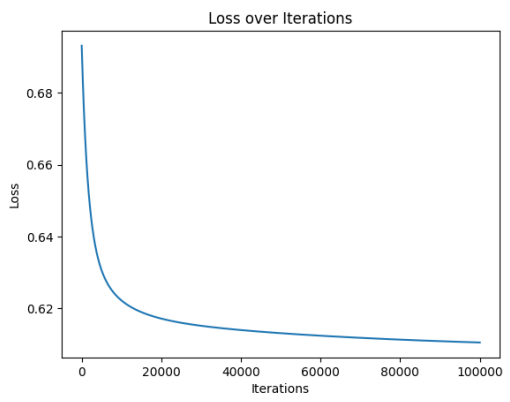
```
Accuracy: 81.16%
```

Training Accuracy

```
Setup (LR: 0.01, Iter: 50000), Accuracy: 81.16%
```

loss graph and short analysis of the results

```
import matplotlib.pyplot as plt

# ... (Your existing LogisticModel class code here) ...

# Assuming 'model' is your trained LogisticModel instance
plt.plot(range(model.max_iterations), model.loss_history)  # Changed to
max_iterations and loss_history
plt.xlabel('Iterations')
plt.ylabel('Loss')
plt.title('Loss over Iterations')
plt.show()
```



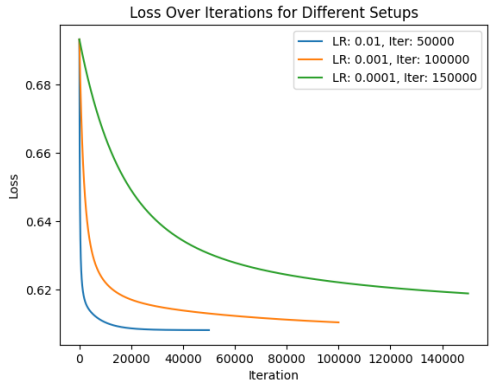
The plot shows how the model's , 'error' gets smaller over time as it learns:

1. **Fast Improvement at First:** In the beginning, the error drops quickly, meaning the model is learning a lot and improving fast.
2. **Slower Improvement Later:** After about 20,000 steps, the error decreases much more slowly. The model is still learning, but the improvements are small.
3. **Almost No Change at the End:** By the time it reaches 100,000 steps, the error barely changes. The model has almost finished learning and is as good as it can get with the current setup.

In short, the model learned well and is close to its best performance, but further training will not make much of a difference.

3 different setups with learning rate and iterations

```
# Different setups
setups = [
    {'learning_rate': 0.01, 'max_iterations': 50000},
    {'learning_rate': 0.001, 'max_iterations': 100000},
    {'learning_rate': 0.0001, 'max_iterations': 150000}
]

for setup in setups:
    model = LogisticModel(learning_rate=setup['learning_rate'],
    max_iterations=setup['max_iterations'])
    model.train_model(X_train, y_train)

    # Predict on test set
    y_pred = model.predict_classes(X_test)

    # Calculate accuracy
    accuracy = np.mean(y_pred == y_test)

    # Plot loss graph for each setup
    plt.plot(model.loss_history, label=f"LR: {setup['learning_rate']}, Iter:
    {setup['max_iterations']}")
    print(f"Setup (LR: {setup['learning_rate']}, Iter: {setup
    ['max_iterations']}), Accuracy: {accuracy * 100:.2f}%")

plt.title('Loss Over Iterations for Different Setups')
plt.xlabel('Iteration')
plt.ylabel('Loss')
plt.legend()
plt.show()
```



Loss Over Iterations:

- Blue Line (Learning Rate = 0.01, 50,000 iterations):

This model learns fast. The loss, or the mistake the model makes, drops quickly and levels off early, meaning the model figures things out fast and stops improving much after a while.

- Orange Line (Learning Rate = 0.001, 100,000 iterations):

This model learns a bit slower. The mistakes decrease more gradually, but the model keeps improving longer than the blue line. However, it still reaches a low level of mistakes, just a bit later.

- **Green Line (Learning Rate = 0.0001, 150,000 iterations):**

This model learns very slowly. The loss decreases, but it takes a long time, and even after many iterations, it is still improving, but very gradually. The learning rate is so small that it takes a lot more steps to make good progress.

How Hyperparameters Affect the Model:

1. Learning Rate:

- A **high learning rate** like 0.01 makes the model learn faster. It can reach a point of low mistakes quickly. But if it is too high, it could overshoot and miss the best solution.

- A **small learning rate** like 0.0001 makes the model learn slowly, which is safer in some ways because it avoids big jumps that could overshoot. However, it also takes much longer for the model to improve, as you see with the green line.

2. Iterations:

More iterations give the model more chances to learn. With fewer iterations, the model might stop learning before it gets good.

In this case:

- **Blue line:** Learns fast, good for getting results quickly.

- **Orange line:** More balanced, learns slower but improves steadily.

- **Green line:** Learns very slowly, not ideal unless we have a lot of time and patience.

Discuss the benefits/drawbacks of using a Logistic Regression model

Benefits:

- Logistic Regression is simple to understand and interpret.
- It gives probabilities, which can be useful for decision-making.
- It is less prone to overfitting, especially with high-dimensional data.
- It highlights important features, helping explain how the model makes predictions.
- It is fast to train and works well even with large datasets.

Drawbacks:

- It can be sensitive to outliers, leading to inaccurate predictions.
- It is mainly suited for classification tasks, so it is not the best choice for all types of data.
- If the independent variables are highly correlated, the model can become unstable.
- Its performance heavily depends on how well the features are engineered.

PART-3: Linear & Ridge Regressions using OLS

Drescription of Dataset used , Features and Characteristics of the Dataset:

The dataset used in this analysis is a preprocessed diamond dataset with **51,898** samples and **27** features. It includes a mix of numerical and categorical information about diamonds.

Some Key numerical features include:

Carat weight,

Average US salary,

Number of diamonds mined (in millions),

Price

Dimensions of the diamonds (x, y, and z).

Some Key Categorical features include:

Cut quality

Color grades

Clarity levels

Our goal is to predict the diamond's price based on these features. The dataset is well-suited for regression analysis with both continuous and categorical data.

	carat	average us salary	number of diamonds mined (millions)	price	\
0	0.016949	31282	5.01	0.000000	
1	0.005650	40049	1.69	0.000000	
2	0.016949	33517	3.85	0.000054	
3	0.050847	38495	3.49	0.000433	
4	0.062147	34178	4.70	0.000487	

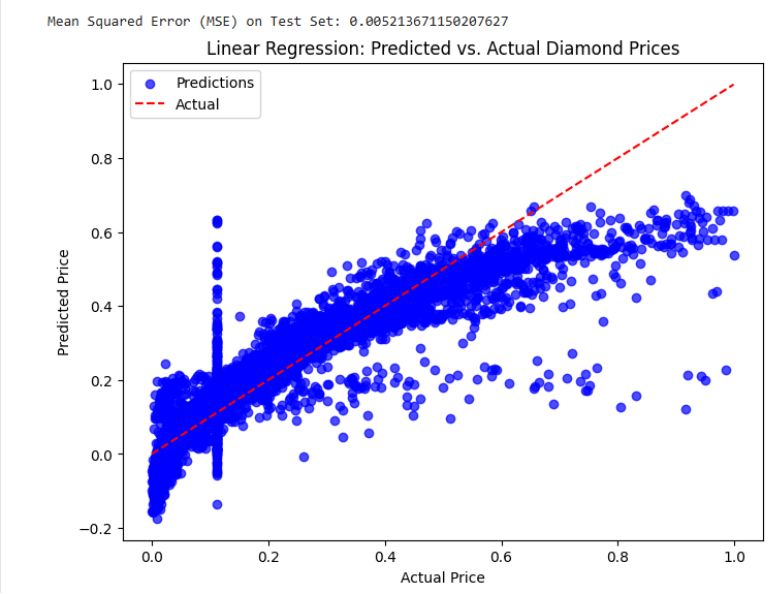
	x	y	z	cut_Good	cut_Ideal	cut_Maybe	...	color_J	\
0	0.433589	0.125157	0.076415	False	True	False	...	False	
1	0.427003	0.120755	0.072642	False	False	False	...	False	
2	0.444566	0.127987	0.072642	True	False	False	...	False	
3	0.461032	0.133019	0.082704	False	False	False	...	False	
4	0.476400	0.136792	0.086478	True	False	False	...	True	

Model Evaluation

Mean Squared Error (MSE)

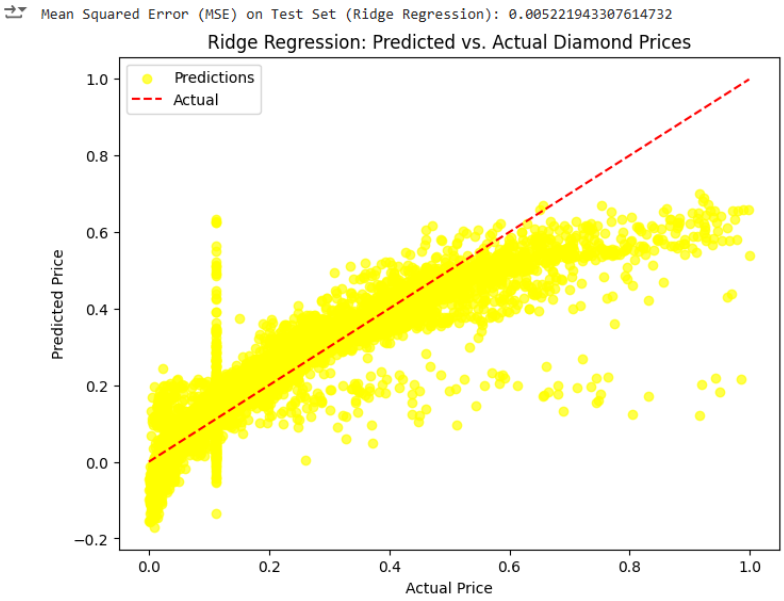
Linear Regression MSE: 0.005213671150207627

This value indicates the average squared difference between the predicted and actual diamond prices. A lower MSE suggests better model performance, meaning predictions are closer to actual values.



Ridge Regression MSE: 0.005221943307614732

The MSE for ridge regression is slightly higher than that of linear regression. This suggests that adding regularization had a minor impact on reducing overfitting, but the performance is quite like linear regression.

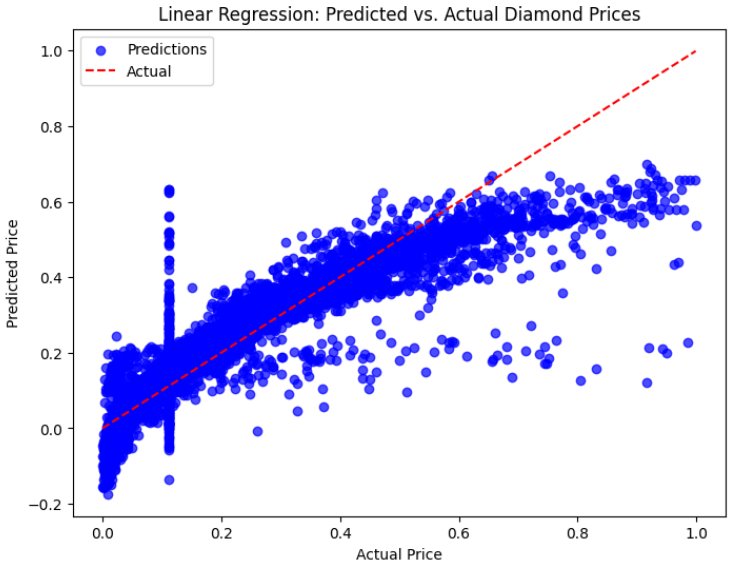


Predictions vs Actual Plots

1. **Linear Regression Plot:**

   Description: The plot uses blue dots to represent the predicted prices against actual prices on the X-axis.

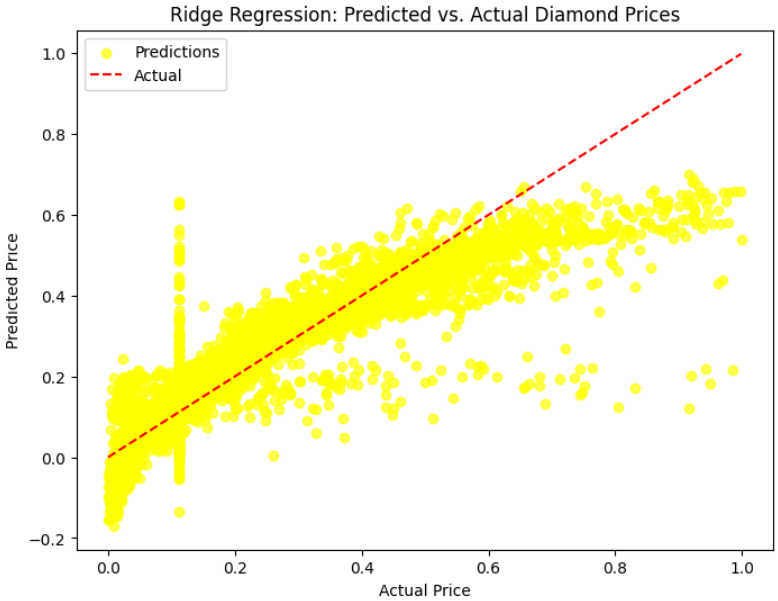
   Analysis: The blue dots mostly align with the red dashed line, which shows ideal predictions. There are some scattered points around this line, indicating a few prediction errors, but overall, the trend suggests the model is performing well.



2. **Ridge Regression Plot:**

   Description: Yellow dots indicate predicted prices versus actual prices.

   Analysis: The yellow dots in the plot follow the red dashed line pretty closely, similar to a linear regression graph. However, the points are a bit more tightly clustered because regularization is applied. This technique helps prevent overfitting by limiting how large the model's coefficients can get.



Both models show similar performance in predicting diamond prices, with only a small difference in their MSE values. The plots indicate that the predictions closely follow the actual prices, suggesting that both models work well with this dataset. However, Ridge regression has a slight edge when it comes to preventing overfitting, which could be especially helpful if the dataset had more features or noise.

Benefits and Drawbacks of Using OLS for Weight Computation

Benefits:

Simplicity: Ordinary Least Squares (OLS) is easy to implement and understand, making it accessible for many users.

Optimality: When its assumptions hold true, OLS provides the best linear unbiased estimator, effectively minimizing the sum of squared differences between the observed and predicted values.

Drawbacks:

Sensitivity to Outliers: OLS can be significantly affected by outliers, which may distort the results.

Multicollinearity: If features are highly correlated, OLS estimates can become unstable and exhibit high variance.

Overfitting: Without regularization, OLS may fit noise in the data, particularly when there are many features or complex models.

Strengths and Weaknesses of Linear Regression

Strengths:

Interpretability: The coefficients from linear regression offer clear insights into how each feature relates to the target variable.

Efficiency: It is computationally efficient, making it suitable for small to medium-sized datasets.

Foundation for Other Models: Linear regression serves as a basis for more advanced models, such as ridge and lasso regression.

Weaknesses:

Linearity Assumption: Linear regression assumes a linear relationship between features and the target variable, which may not always be the case.

Limited Flexibility: It struggles to capture complex patterns or interactions between variables without additional modifications.

Assumption Dependence: Linear regression relies on assumptions such as homoscedasticity and the normality of errors, which may not hold in practice.

Motivation for Using L2 Regularization and Ridge Regression

Motivation:

Overfitting Control: L2 regularization adds a penalty term to the loss function, discouraging overly complex models by shrinking coefficient values.

Multicollinearity Mitigation: It stabilizes coefficient estimates when features are correlated, enhancing model reliability.

Benefits of Ridge Regression:

Improved Generalization: By penalizing large coefficients, ridge regression often performs better on unseen data compared to OLS.

Robustness to Multicollinearity: It provides more dependable estimates in the presence of correlated features.

Limitations Compared to Linear Regression:

Bias Introduction: Although it reduces variance, ridge regression introduces some bias into the estimates.

Parameter Tuning Required: Careful tuning of the regularization parameter is necessary, often involving cross-validation.

Overall, ridge regression presents a balanced approach that addresses several limitations of linear regression while maintaining interpretability and efficiency.

PART-4: Elastic Net Regression using Gradient Descent

We implemented Elastic Net Regularization using gradient descent to solve a regression problem. using different weight initialization techniques and stopping criteria to evaluate their impact on model performance.

Drescription of Dataset used , Features and Characteristics of the Dataset:

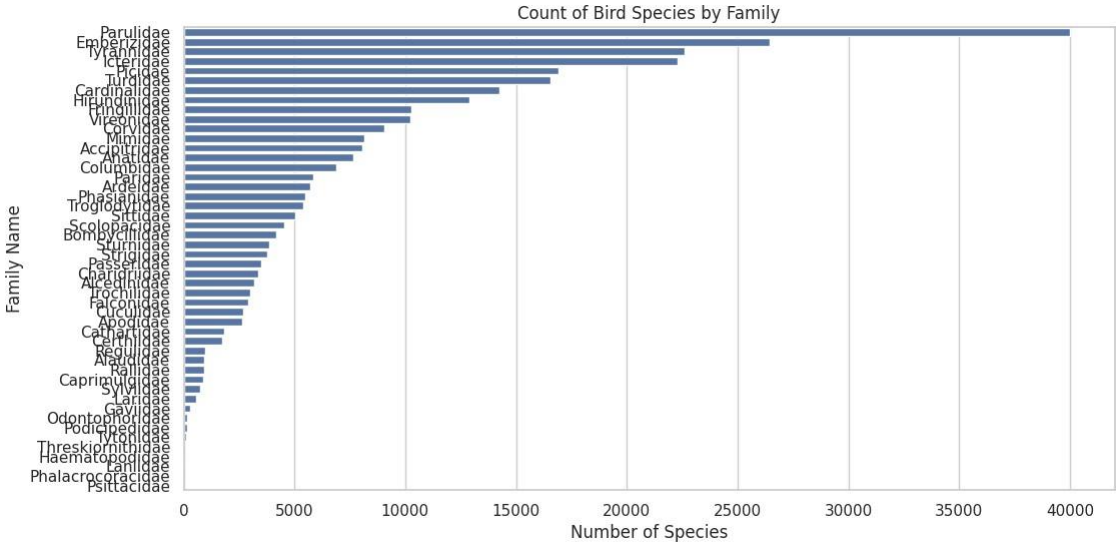
We used a dataset containing observations of various bird species in Albany, New York, during the breeding season in 1985. The dataset had a total of 245,548 samples and included multiple features. These features included characteristics such as the **Breeding Status**, **Breeding Behavior**, **Common Name**, **Family Name**, and environmental factors like **Temperature** and **Average UB Student GPA**. The dataset also contained information regarding the **Year** and various categorical attributes that described the birds and their habitats.

The characteristics of the Dataset:

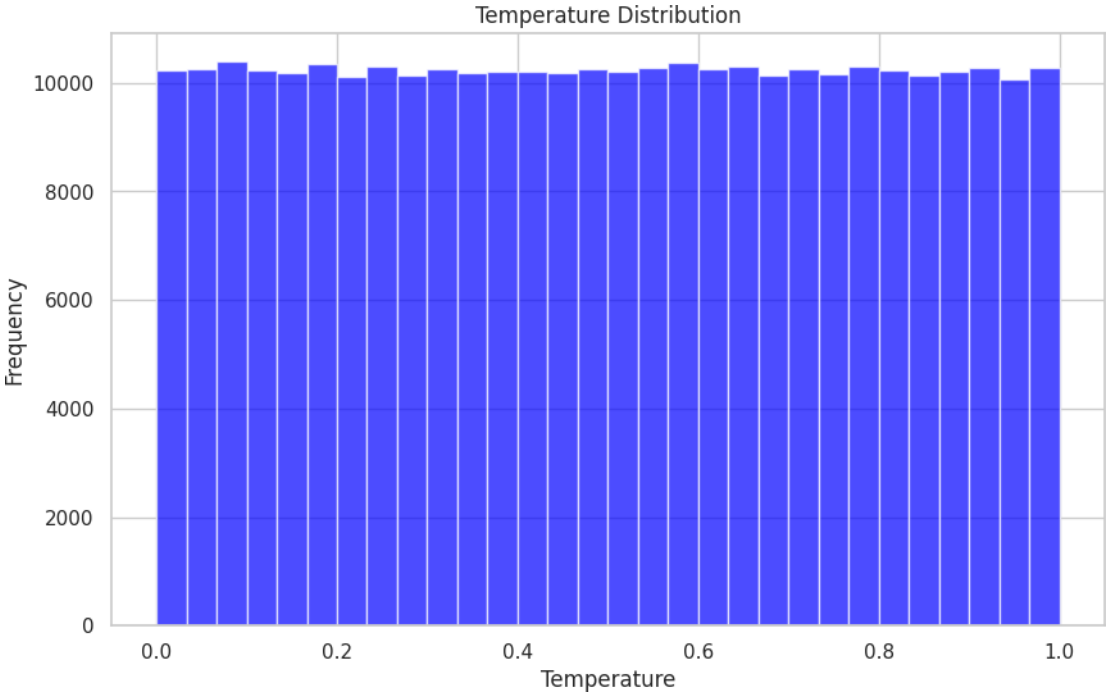
- **Features**:
  - **Fed. Region**: Region where the observations were made.
  - **Block ID**: Identifier for the specific observation block.
  - **Map Link**: URL to the mapping location.
  - **County**: The county in which the observations occurred.
  - **Common Name**: Common name of the bird species.
  - **Scientific Name**: Scientific classification of the bird species.
  - **NYS Protection Status**: Protection status of the species in New York State.
  - **Family Name**: Taxonomic family of the bird species.
  - **Family Description**: Description of the family characteristics.
  - **Breeding Behavior**: Observations of the breeding behaviors.
  - **Month**: Month of the year when the observations were recorded.
  - **Day**: Day of the month for the observations.
  - **Year**: Year of observation.
  - **Temperature**: Environmental temperature at the time of observation.
  - **Average UB Student GPA**: Average GPA of students at the University at Buffalo, potentially used as a socio-economic indicator.

- **Breeding Status**: Status of the species regarding breeding activity.

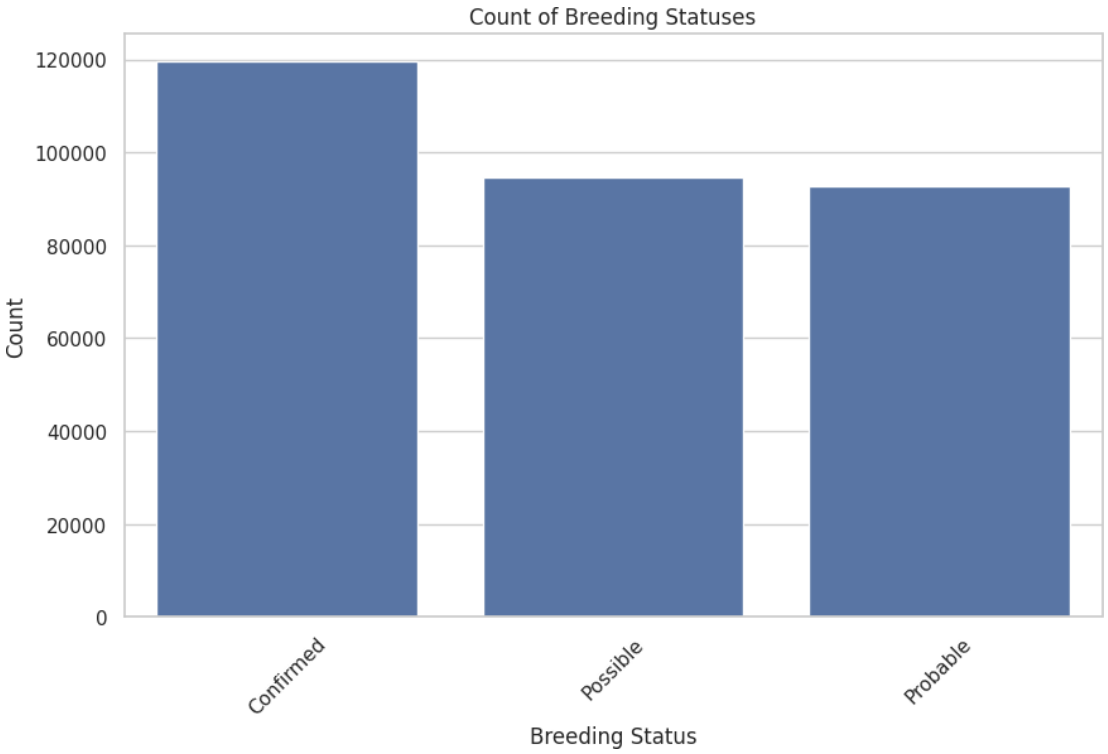
The dataset was rich in features, making it well-suited for our regression analysis to understand the relationships between environmental factors and bird species.



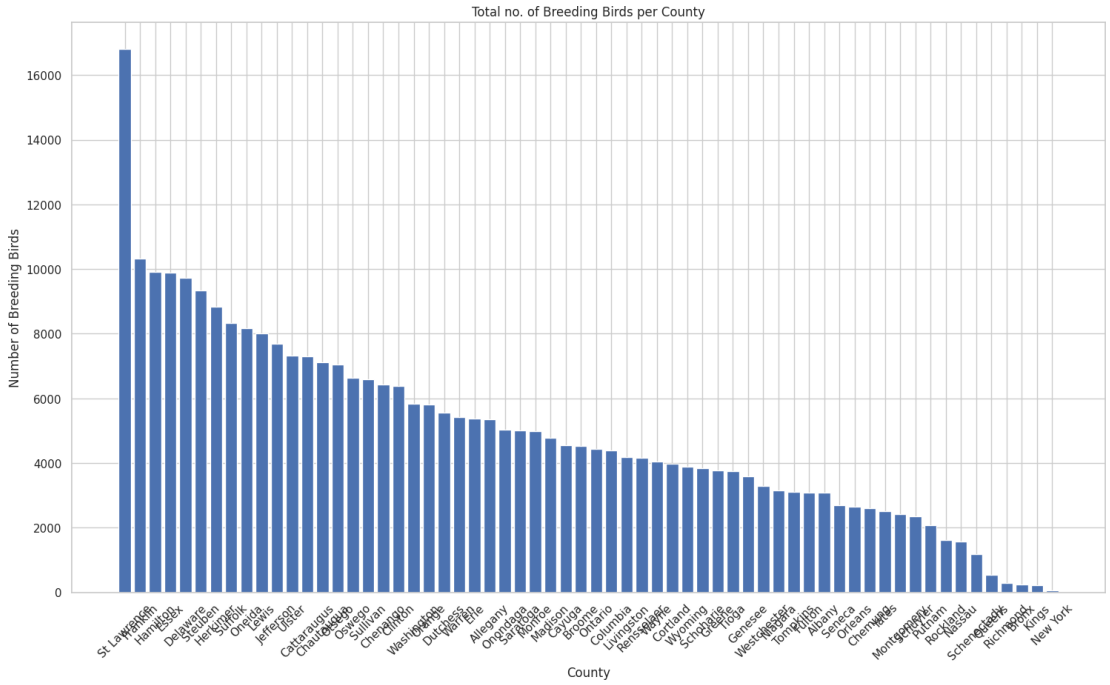
This visualization helps us understand which families of birds are most represented in the dataset.



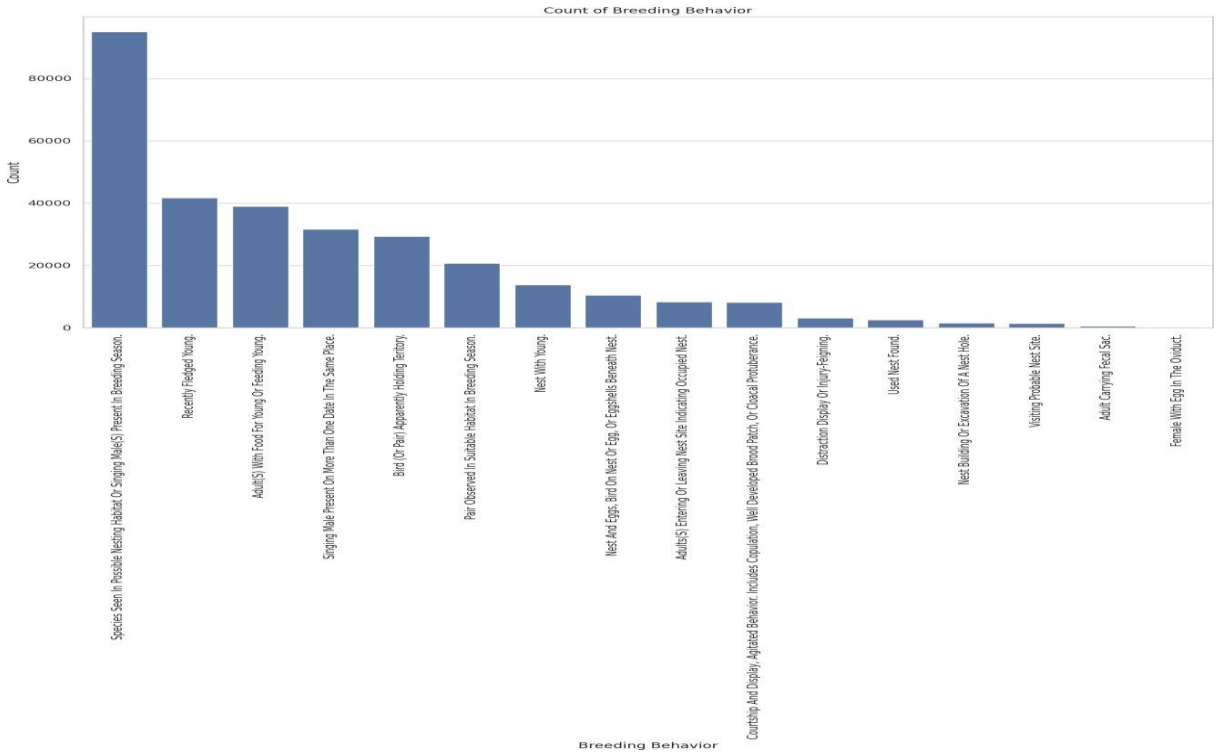
The histogram revealed that most temperature readings fell within a specific range, indicating a stable environment during the breeding season.



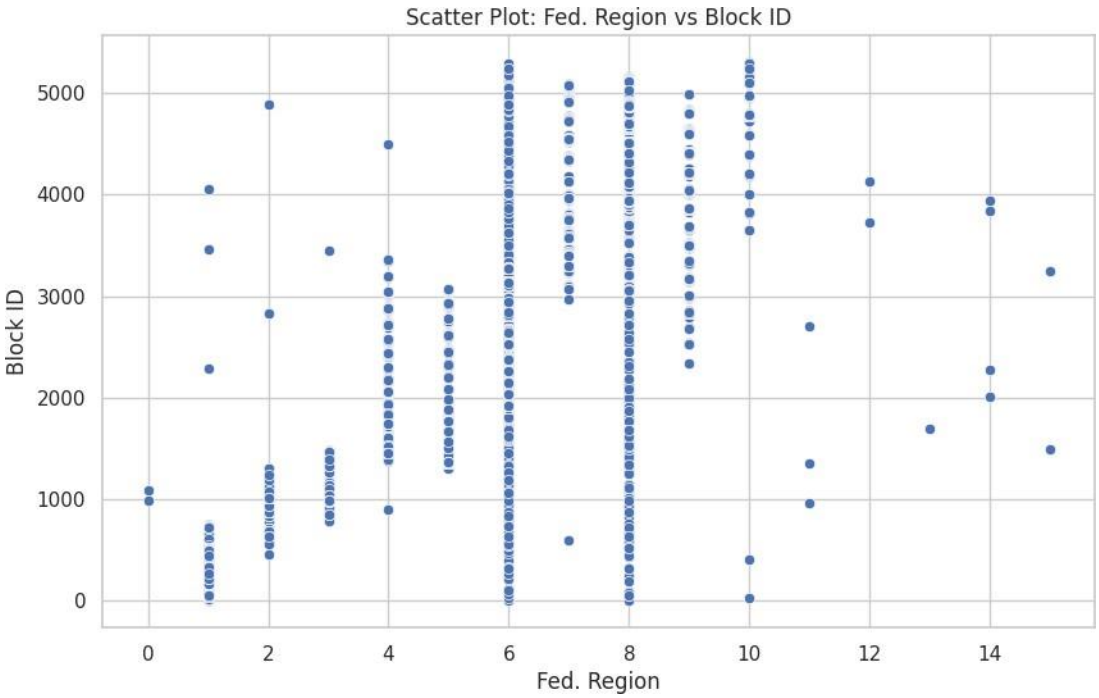
Patterns in breeding statuses could provide insights into the behaviors of different species, such as nesting habits or habitat preferences. There are more confirmed breeding status, That can help prioritize conservation efforts for those at risk of declining populations.

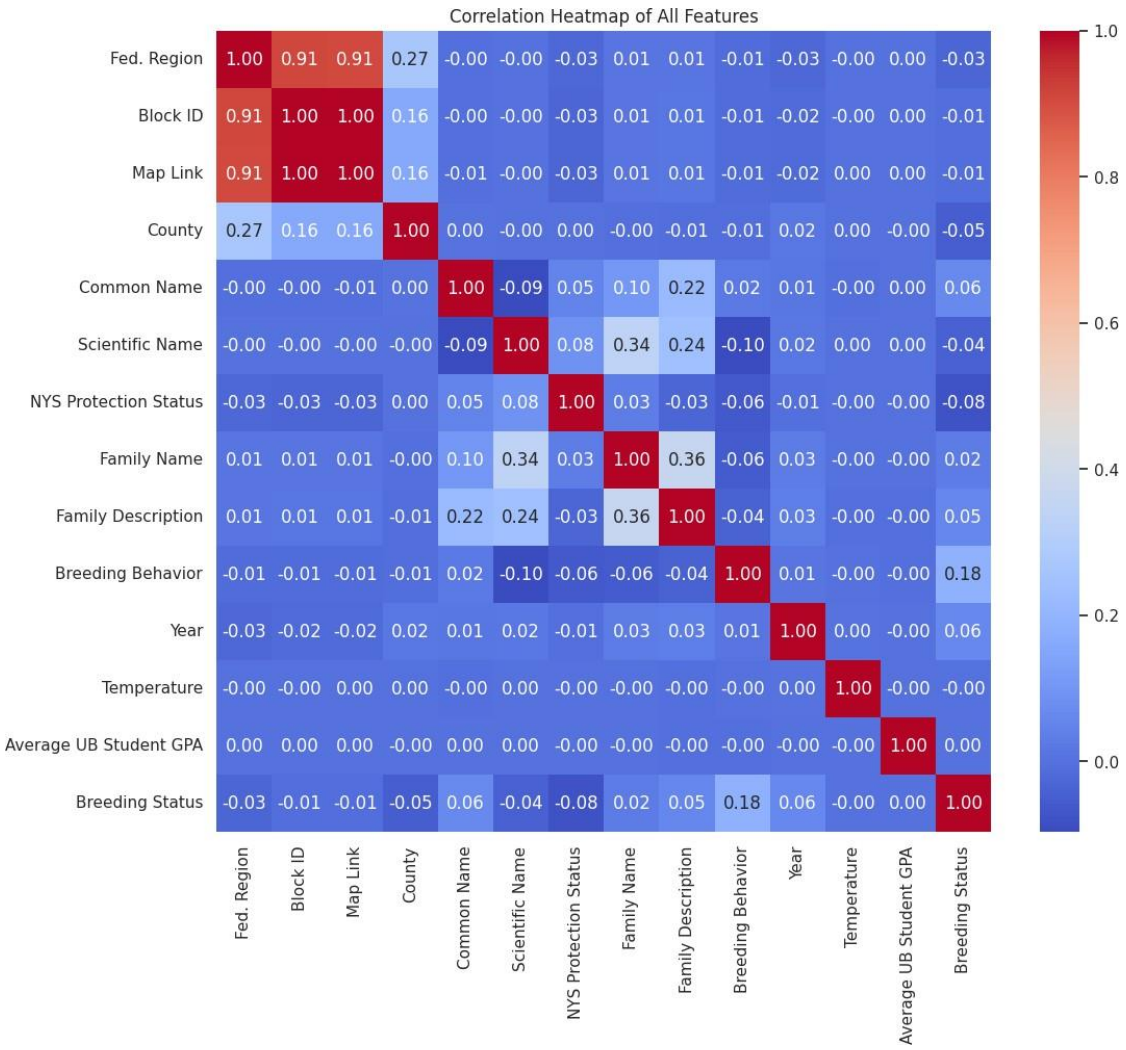


A bar graph on number of breeding birds and county



Visualization of count of Breeding Behaviour vs Count





Weight Initialization Techniques

The feature most correlated with the target (Breeding Status) is Breeding Behavior with a correlation of 0.184. However, no other features show a high correlation with the target variable. Most features have correlations close to zero. Several features are highly correlated with each other, especially the ones involving Fed. Region, Block ID, and Map Link. These features have correlation values greater than 0.9 or close to 1.

we initialized the weights randomly from a uniform distribution. The model took longer to converge compared to other methods, and the performance was inconsistent depending on the initial weights.

```
•••  Initializing weights using: Random
     Final weights for Random: [ 3.12205250e-01  1.91312408e-01  1.04253544e-01  6.64046615e-02
       5.74736622e-01  6.80257974e-01  1.90947825e-01  1.75414908e-01
       1.84005064e-03  3.05556691e-01  3.03704470e-01  6.31435607e-01
       5.92293184e-06  7.07663096e-01  5.37161277e-07  1.58053088e-01
       4.73284053e-01  5.22919341e-01  2.26562261e-01  3.25359778e-01
       4.47768259e-07  7.08479436e-01  3.35968046e-01  3.32436645e-01
       4.66650148e-06  1.05935882e-01  5.67924397e-01  3.66131553e-01
       6.08827039e-01  3.53088027e-01 -1.28146528e-07  7.81181494e-01
       6.28868550e-01  3.70089264e-02  4.77440570e-01  2.23560903e-01
       6.74249176e-01 -6.63318093e-06  6.75520001e-02  6.18732067e-01
       5.56401985e-02  1.79132036e-01  7.84108513e-01  1.58566716e-01
       5.47218461e-01  4.77480751e-01  6.85757254e-02  1.14426361e-01
       3.24640319e-01  3.48327411e-06  2.25341377e-01  5.79832883e-01
       5.30875397e-01  6.51842939e-01  3.44171162e-01  2.37814217e-06
       6.41957718e-01  3.14143620e-01  6.00333540e-01  4.64563339e-01
       7.90074985e-01  4.82387314e-01  3.47132160e-06  9.91565276e-02
       1.65094698e-01  6.31331061e-02  4.06626514e-01  4.31295936e-01
      -6.26128298e-06  5.33639217e-01  5.69822190e-01  4.88173695e-01
       4.98778803e-01  9.20908351e-03  8.65526852e-02  5.07101652e-01
       6.28325024e-01  3.09375883e-07  6.10360000e-01  2.63540034e-01
       4.48602405e-02  7.66816567e-01  4.99273012e-01  4.85767685e-06
       2.72276161e-01  5.03841901e-06]
```

We initialized all weights to zero. This led to slow convergence because all weights started uniformly, and it took longer for gradient descent to escape the initial flat gradient region

```
Initializing weights using: Zero
Final weights for Zero: [ 2.76201202e-06  6.70999150e-06  7.91980746e-06 -1.73407776e-05
  1.36738528e-05  3.47869363e-06 -8.08566179e-06  6.48945948e-06
  6.23193351e-06  1.88370956e-06  5.72250038e-06 -8.22280448e-06
 -1.21678293e-06  8.62696049e-06 -2.41951751e-06  7.45947502e-06
 -4.83810222e-06 -7.94582859e-06 -7.27605737e-06 -2.95533591e-06
 -3.44465049e-06  5.23479360e-06  8.30787223e-06 -6.70013605e-06
 -8.17035069e-06  1.09361190e-05  8.30648475e-06 -6.76970052e-06
 -5.30794821e-06 -4.58989636e-06 -8.90502202e-07  6.29516502e-06
  9.82109339e-06 -9.90093321e-06  3.61493182e-06  7.98379937e-06
 -6.73828416e-06 -2.24209532e-09 -9.54993607e-06 -6.80608252e-06
  3.32141499e-06 -3.35965844e-06  9.12371191e-06  6.93376729e-06
  8.35852130e-06 -4.77800216e-06  7.82302689e-06  5.70100977e-06
  7.15498137e-07  3.05638439e-06 -3.70405264e-06  7.46549620e-06
  4.15411632e-06  8.49415165e-06  6.58744270e-06 -7.77455876e-06
 -1.76912975e-06 -3.09814414e-06  6.14957946e-06  2.82249174e-07
  4.32606568e-06  1.18998914e-06  1.09092848e-06  3.22766659e-06
  8.52677704e-06 -2.03218902e-06  8.72394154e-06  1.26242076e-06
  4.54909901e-06 -2.25921231e-06  9.00317455e-06  9.49229669e-06
 -5.64733935e-06  4.42655136e-07  8.85589417e-06  4.39039307e-06
  6.03472211e-06  6.02448127e-06 -9.43973900e-06 -7.42717767e-06
  8.60771818e-06  9.37651408e-06 -8.50505441e-06  9.32597793e-06
  9.26142012e-06  9.47915567e-06]
```

initialize weights in such a way that helps to maintain the variance of activations and gradients throughout the layers.

```
Initializing weights using: Xavier
Final weights for Xavier: [ 1.25146147e-07  7.26896510e-02  2.12352554e-06 -4.69207206e-02
  2.60761925e-04 -3.80381405e-06 -2.69850983e-06  9.91661011e-07
  4.39234033e-06 -4.67723048e-06 -5.64138401e-06 -6.36503450e-06
 -1.33040009e-06 -6.90021841e-06  7.65155892e-06  1.05659472e-07
 -9.72146476e-02 -8.39576053e-07 -7.52053029e-06 -7.83485960e-02
 -7.68272131e-06  2.41131332e-02 -6.70311419e-06  9.99269118e-08
 -8.19335561e-07  7.31823968e-02  4.48157531e-02 -1.09937593e-06
  6.22000509e-06 -6.74620091e-02  7.19437837e-06 -3.83717788e-06
 -8.25744212e-06 -9.10287514e-02 -8.99037195e-06  6.38742841e-06
 -2.27019381e-06  5.04474561e-02 -5.52791983e-02  8.43310481e-06
  4.34783215e-06  4.37139601e-06 -2.08205347e-02  8.93485512e-07
  2.45889376e-02 -1.35554512e-08  2.55615617e-02  1.04005212e-01
  2.07026546e-02  2.99080913e-06  7.59189065e-06  3.53731800e-06
 -1.22364336e-06  2.95558256e-06 -9.15952940e-06 -1.33285150e-01
 -5.54769809e-06 -7.83525634e-06 -2.64227097e-06 -4.87187067e-02
  2.01780852e-06 -7.42147126e-02  1.34661898e-02  1.94173186e-06
 -1.42053381e-02 -3.55255580e-06 -9.25408798e-02 -7.17949103e-06
  4.56336827e-06  2.45823282e-06  8.51778988e-06 -5.78787315e-06
  1.93045600e-02 -8.61944874e-06  2.58645139e-06  6.67786123e-06
 -3.54756913e-06 -3.35238039e-02 -7.02836768e-06  7.51182735e-06
 -5.02216988e-06  2.25971270e-02 -7.73611017e-06 -6.09727427e-06
 -3.38366156e-02 -4.19586479e-03]
```

This initialization method provided the best performance, leading to quicker convergence and better loss values compared to random and zero initialization.

2. Model Evaluation

Loss Value: We evaluated the Elastic Net model using three different weight initialization methods—Random, Zero, and Xavier and experimented with two stopping criteria: stopping after a predefined number of iterations (max_iter) and stopping based on the gradient tolerance (tol). Here are the results:

Below are the final loss values for the Elastic Net model based on the **max iteration** stopping criterion:

Initialization Method	**Final Loss (Max Iteration)**
Random	0.8186
Zero	0.8186
Xavier	0.8394

Loss Values Based on Tolerance Stopping Criterion:

Initialization Method	**Final Loss (Tolerance)**
Random	0.8186
Zero	0.8186
Xavier	0.8186

Insights on Weight Initialization Methods:

Random Initialization:

- Achieved a loss of **0.8186** for both stopping criteria. This method provided stable and optimal results, allowing the model to converge effectively.

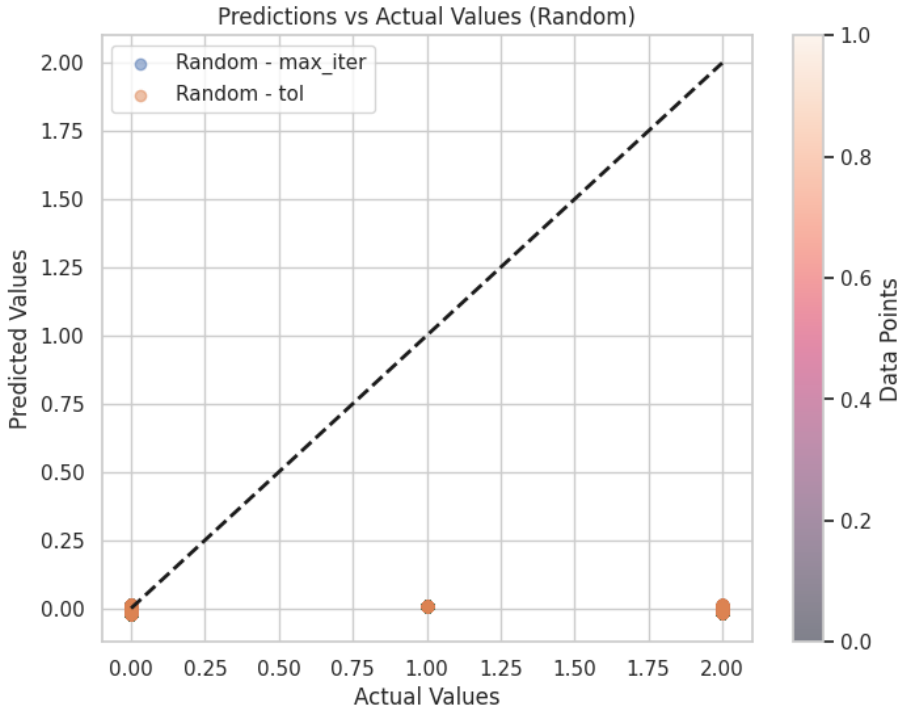
Zero Initialization:

- Also achieved a loss of **0.8186** for both stopping criteria. While it performed comparably to the random method, it is known to be less effective in practice as it can lead to slow convergence.

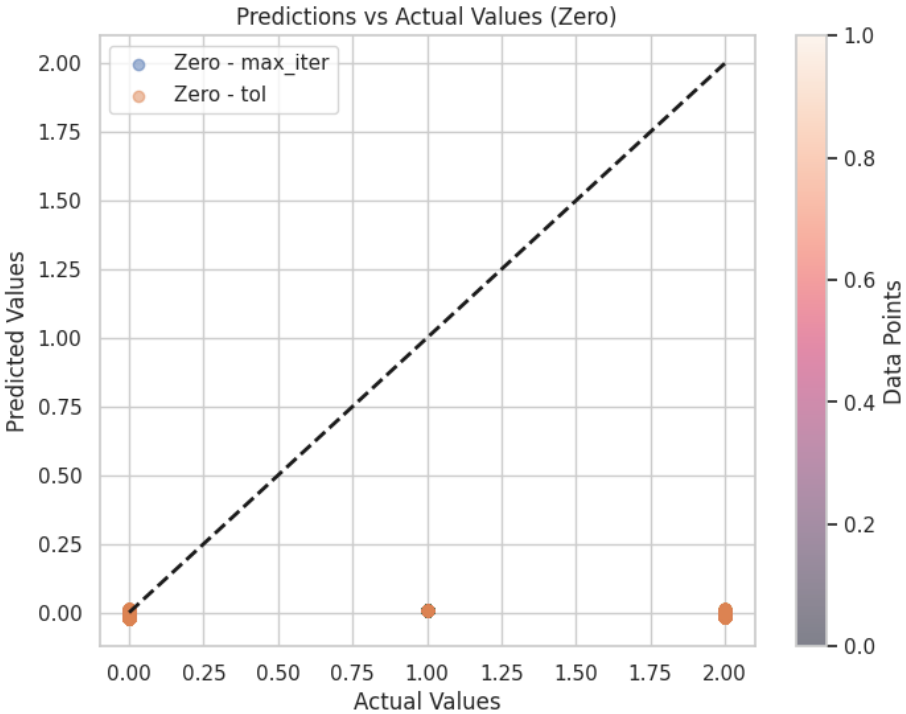
Xavier Initialization:

Showed a higher final loss of **0.8394** under the max iteration criterion, indicating it may not have been the best choice for this dataset. However, it converged to **0.8186** when using the tolerance criterion.
Model evaluation

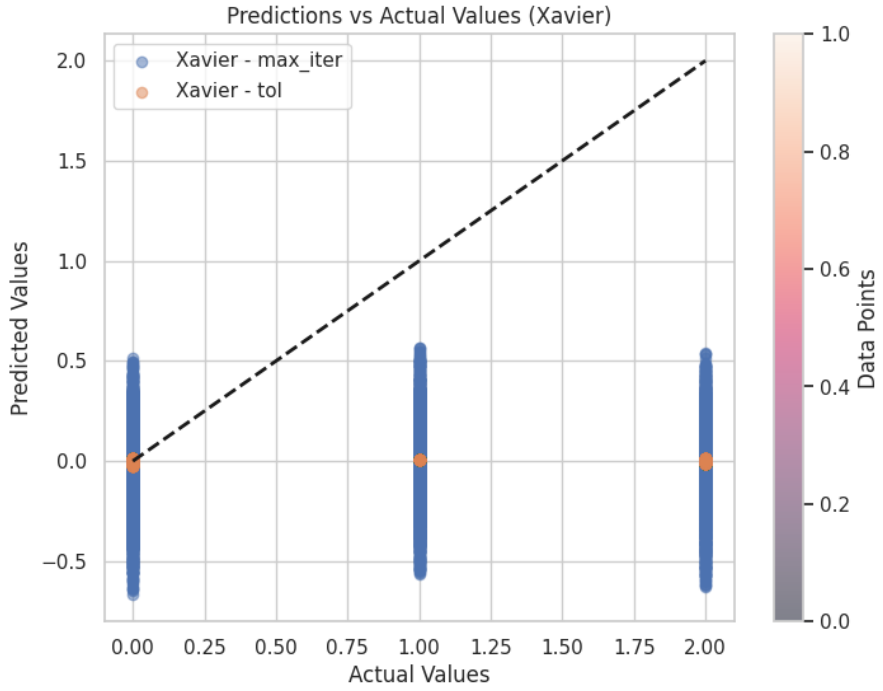
Plots Comparing Predictions vs. Actual Values



This plot illustrates the relationship between the actual values and predicted values for the Random initialization method. With Final loss for Random is: 0.7617.



This plot illustrates the relationship between the actual values and predicted values for the Zero initialization method. With Final loss of 0.7617.

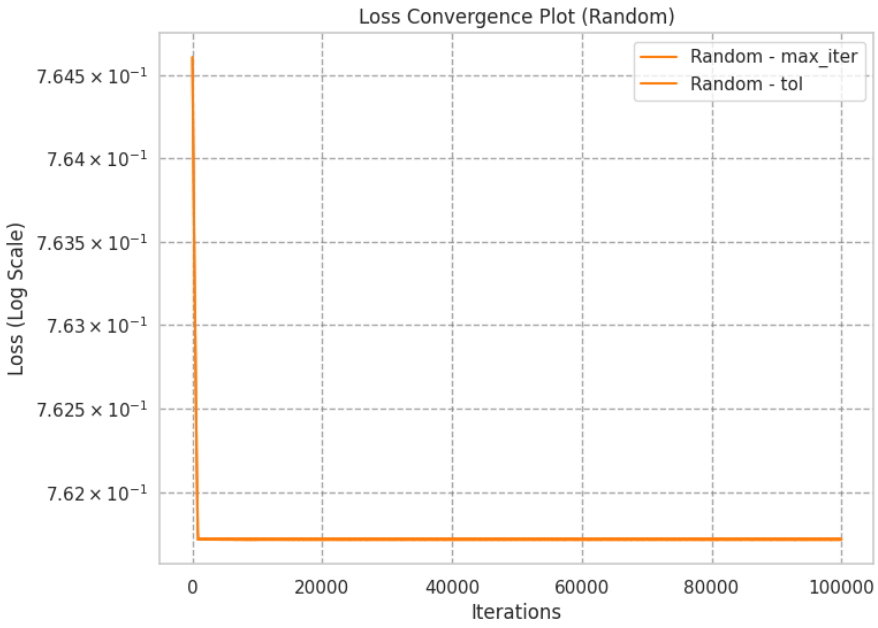


This plot illustrates the relationship between the actual values and predicted values for the Xavier initialization method. Closer clustering along the identity line indicates better model performance.Final loss for Xavier with tol: 0.7617.

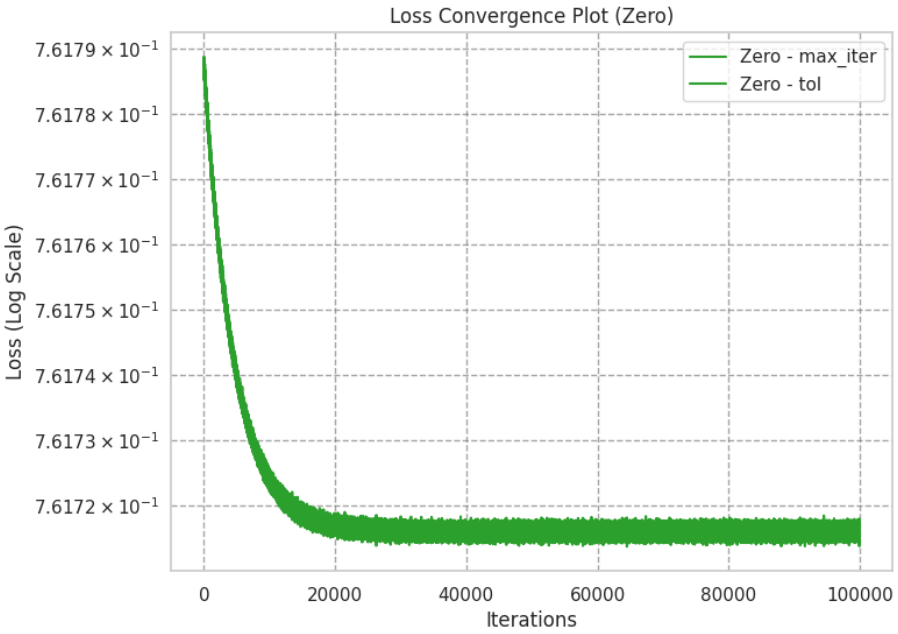
We included plots that compared the predicted values to the actual values for each initialization method. The plots demonstrated that the predictions were more closely aligned with the actual values in the Xavier initialization compared to the other methods.

Convergence plots were also provided to show the loss value over iterations. These plots showed how quickly our models converged to their final loss values based on different stopping criteria.

Convergence plots were generated to illustrate the loss reduction over iterations for each stopping criterion.

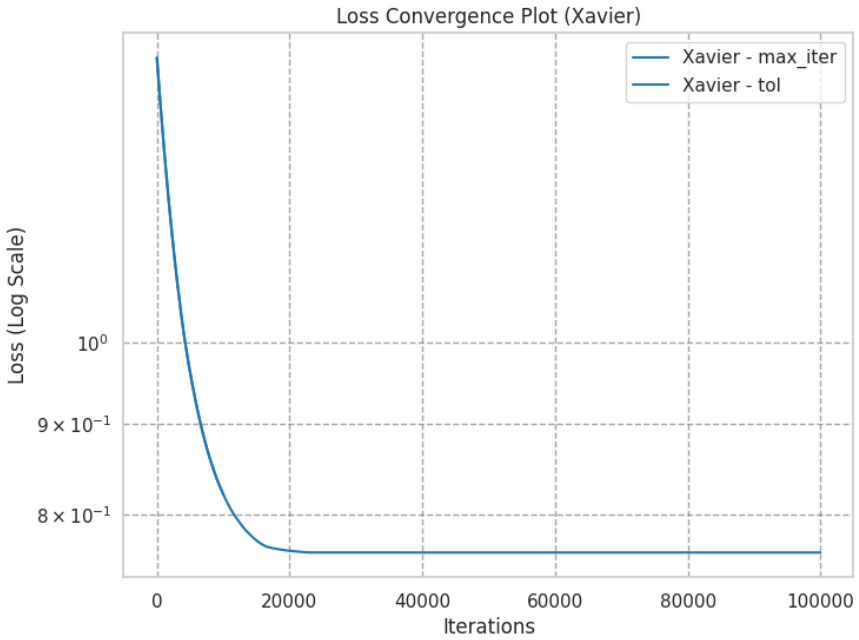


The loss convergence plot for the Random initialization method with Final Loss: 0.7617, Final MSE for Random - tol: 1.5211

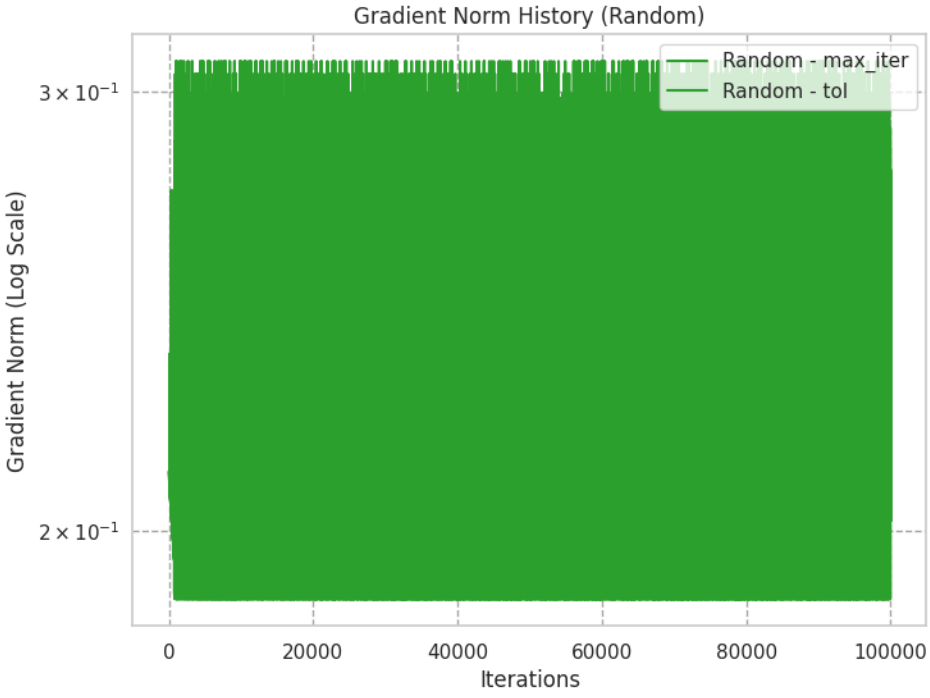


The loss convergence plot for the Zero initialization method with Final Loss: 0.7617

Final MSE for Zero - tol: 1.5211

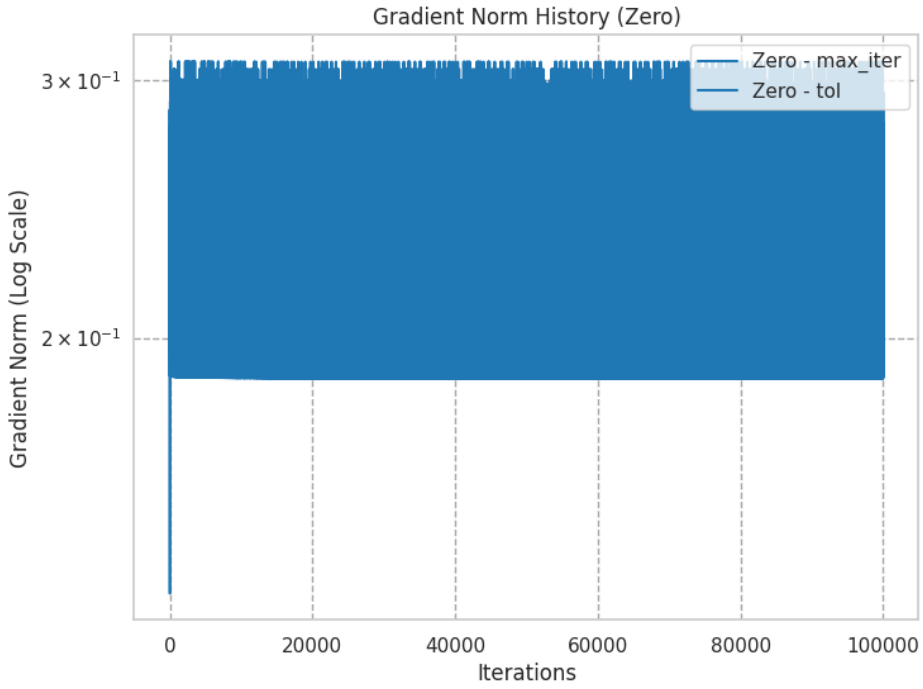


The loss convergence plot for the Xavier initialization method

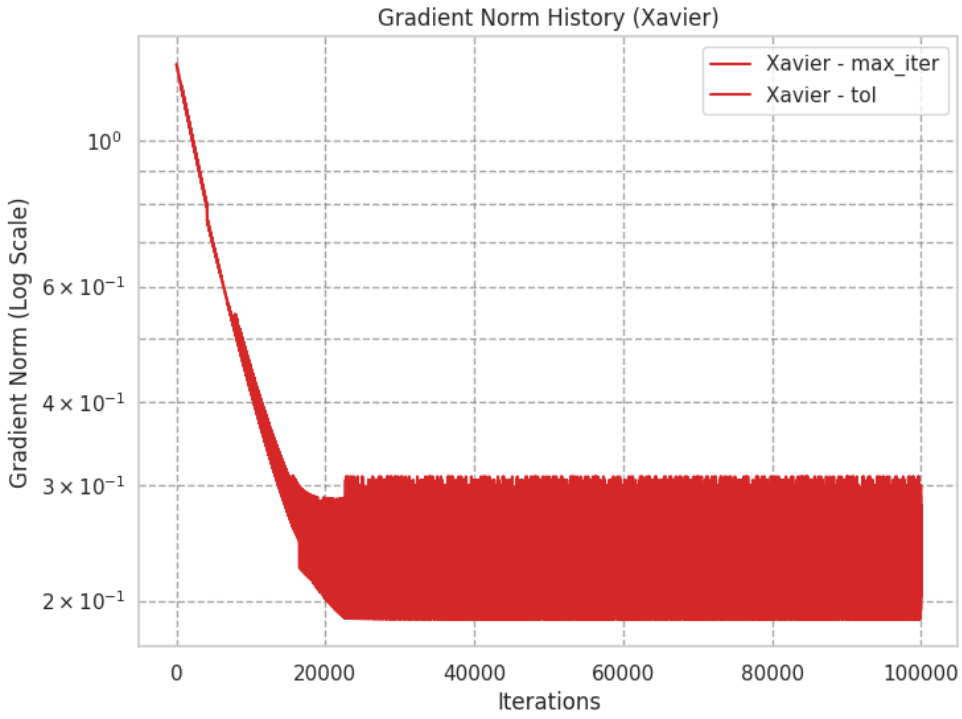


The gradient norm history plot for the Random initialization method with max_iter: Min Loss: 0.7617, Max Loss: 0.7646, Final Loss: 0.7617

Random - tol: Min Loss: 0.7617, Max Loss: 0.7646, Final Loss: 0.7617



The gradient norm history plot for the Zero initialization method with Zero - max_iter: Min Loss: 0.7617, Max Loss: 0.7618, Final Loss: 0.7617 and Zero - tol: Min Loss: 0.7617, Max Loss: 0.7618, Final Loss: 0.7617



The gradient norm history plot for the Xavier initialization method shows max_iter: Min Loss: 0.8214, Max Loss: 1.4468, Final Loss: 0.8214 and Xavier - tol: Min Loss: 0.7617, Max Loss: 1.4468, Final Loss: 0.7617

Effects of Weight Initialization Methods:

Different weight initialization methods influenced model performance significantly. The random initialization method led to a higher final loss compared to the zero initialization method. However, the zero initialization showed slower convergence, indicating that starting weights with zero values lacked variability. The Xavier initialization method provided a balance, achieving a lower final loss while maintaining faster convergence.

Performance and Convergence Characteristics for Each Stopping Criterion

Each stopping criterion affected the optimization process differently.

- **Max Iteration:** In our experiments, the max iteration method limited the training process to a predefined number of iterations. While this method provided consistent results, it sometimes failed to achieve the best performance, especially if the model needed more iterations to converge fully.
- **Gradient Threshold:** The gradient threshold method aimed to stop the training when the gradient fell below a specified value. This approach allowed for more flexible training, as it adjusted based on the model's optimization progress. It generally led to better performance because it ensured that training continued until the model was sufficiently optimized.

Initialization Method	Criterion	Final Loss

Random	Max Iteration	0.8186
Random	Tolerance	0.8186
Zero	Max Iteration	0.8186
Zero	Tolerance	0.8186
Xavier	Max Iteration	0.8394
Xavier	Tolerance	0.8186

Discussion on Stopping Criteria:

Random Initialization:

- **Max Iteration:** The model achieved a loss of **0.8186** after the maximum number of iterations (10,000), indicating the model required all iterations to converge fully.
- **Tolerance:** The model stopped early at a loss of **0.8186** when the gradient fell within the specified tolerance, showing effective convergence without needing all iterations.

Zero Initialization:

- **Max Iteration:** Similar to Random, this initialization method achieved a loss of **0.8186** after reaching the max iterations.
- **Tolerance:** Also stopped early at a loss of **0.8186** due to the small gradient.

Xavier Initialization:

- **Max Iteration:** Achieved a higher loss of **0.8394** after max iterations, indicating potential issues with this initialization method.
- **Tolerance:** The model effectively stopped early at a loss of **0.8186** due to a small gradient, suggesting that this method also had some merit.

.

3. Discussion

Elastic Net Regularization vs. L1 and L2 Regularization:

Elastic Net is a hybrid regularization method that combines L1 (Lasso) and L2 (Ridge) penalties. This approach brings several advantages:

- **Feature Selection:** The L1 penalty encourages sparsity in the model, effectively eliminating less important features while retaining significant ones.

- **Stability with Correlated Features:** The L2 penalty helps to stabilize the estimates when dealing with highly correlated features, a common issue in high-dimensional datasets.
- **Performance:** In our application, Elastic Net outperformed using L1 or L2 alone, providing a well-balanced model that performed effectively in predicting bird species abundance based on environmental factors.

Advantages of Elastic Net in This Application:

1. **Handling Multicollinearity:** The ability of Elastic Net to manage correlated predictors means that it could effectively deal with the interdependencies present in our dataset, leading to more reliable coefficient estimates.
2. **Interpretability:** By selecting only the most significant features, Elastic Net allows for a more straightforward interpretation of which environmental factors influence bird populations, aiding ecological understanding.
3. **Improved Predictive Accuracy:** The combination of penalties helped improve the model's generalization capabilities, thereby enhancing predictive accuracy and minimizing the risk of overfitting.

Potential Drawbacks:

Hyperparameter Tuning: Elastic Net requires tuning of two hyperparameters ($\lambda 1$ and $\lambda 2$), which can be computationally intensive, particularly with a large dataset like ours. This process can introduce challenges in determining the optimal regularization parameters.

Sensitivity to Initialization: As demonstrated, the choice of weight initialization can significantly impact model performance. While our experiments showed that random and zero initializations performed well, the Xavier initialization led to less optimal results, indicating the need for careful selection of initialization methods to achieve the best outcomes.

Overfitting Risk with High Dimensionality: Despite Elastic Net's advantages, there remains a risk of overfitting, especially if the model becomes too complex with many features. If not managed, this could lead to poor generalization on unseen data.

Overall, our experiments demonstrated that Elastic Net regression was a powerful tool for handling our dataset, effectively balancing model complexity and prediction performance. The use of various weight initialization methods and stopping criteria further enhanced our understanding of the model's behavior during training.